

Choosing the Right Database Management System

Selection Criteria, Database Types, and Operational Models

Lecture Notes

Course: Databases

Updated: 31/05/2026

Learning Objectives

After completing this lesson, learners will be able to:

1. Understand why choosing a database management system is an important architectural decision.
2. Distinguish common database types: relational, document, key-value, time-series, columnar, and graph databases.
3. Identify the main criteria for selecting a DBMS for a software system.
4. Understand when to use a self-managed database and when to use a managed database service.
5. Apply the knowledge to choose a suitable database type for a practical problem.
6. Complete review questions and application exercises on DBMS selection.

Lecture Structure I

- 1 Overview
- 2 Basic Concepts
- 3 Criteria for Selecting a DBMS
- 4 Common Database Types
 - Relational Database
 - Document Database
 - Key-Value Store
 - Time-Series Database
 - Columnar Database
 - Graph Database
- 5 Database Management and Operations
 - Managed Database
 - When Should We Choose a Managed Database?
 - When Should We Self-Manage a Database?
- 6 Architectural Patterns Affecting Database Choice
 - Polyglot Persistence

Lecture Structure II

- CQRS
- Event Sourcing
- Microservices

7 Comparison and DBMS Selection Process

- Comparison Table
- Five-Step Selection Process

8 DBMS Selection Scenarios

- Student Management System
- Social Network Application
- IoT Cold-Storage Temperature Monitoring
- E-Commerce System

9 Implementation Notes and Summary

10 Questions and Exercises

Section Outline

- 1 Overview
- 2 Basic Concepts
- 3 Criteria for Selecting a DBMS
- 4 Common Database Types
- 5 Database Management and Operations
- 6 Architectural Patterns Affecting Database Choice
- 7 Comparison and DBMS Selection Process
- 8 DBMS Selection Considerations

Why Does Choosing the Right DBMS Matter?

In any software system that uses data, selecting the right **database management system** is a very important decision.

A suitable choice can help the system:

- Store data safely and consistently.
- Query data faster.
- Scale more easily as the number of users grows.
- Reduce operational costs.
- Become easier to maintain and evolve in the future.

Consequences of Choosing the Wrong DBMS

If the wrong database type is selected, the system may face many problems:

- Slow queries.
- Difficulty scaling.
- Inconsistent data.
- High operational costs.
- Difficulty changing the data structure as the system evolves.

Key Message

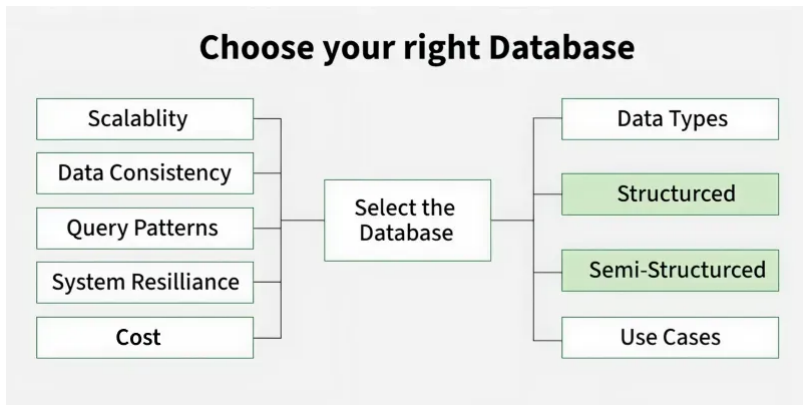
Do not choose a database simply because it is popular. The choice should depend on the nature of the data, query patterns, performance, consistency requirements, scalability, and operational capability.

Factors to Consider When Choosing a Database

The decision should be based on:

- The data type used by the system.
- How the data is queried.
- Performance requirements.
- Consistency requirements.
- Scalability requirements.
- Operational capability of the technical team.
- Cost.
- Long-term system development direction.

Figure 1. Choosing the Right Database



Overview of choosing an appropriate database.

Quick Quiz: Overview

Question 1. Why should we not choose a DBMS only because it is popular?

- A. Because popular DBMSs are always free
- B. Because DBMS selection should depend on data, queries, performance, and operations
- C. Because popular DBMSs do not support security
- D. Because every system should use only one database type

Question 2. What problem may occur when the DBMS choice is unsuitable?

- A. Slow queries and difficulty scaling
- B. Higher monitor resolution
- C. Fewer tables required in every system
- D. Automatic removal of the need for backups

Section Outline

- 1 Overview
- 2 Basic Concepts**
- 3 Criteria for Selecting a DBMS
- 4 Common Database Types
- 5 Database Management and Operations
- 6 Architectural Patterns Affecting Database Choice
- 7 Comparison and DBMS Selection Process
- 8 DBMS Selection Considerations

What Is a DBMS?

DBMS stands for **Database Management System**.

A DBMS is software used to:

- Create databases.
- Store data.
- Query data.
- Update data.
- Delete data.
- Ensure data security.
- Ensure data integrity.
- Manage access and transactions.

Examples of DBMSs

- MySQL
- PostgreSQL
- SQL Server
- Oracle Database
- MongoDB
- Redis
- Neo4j
- Snowflake

Quick Quiz: Basic Concepts

Question 1. What does DBMS stand for?

- A. Database Management System
- B. Data Backup Monitoring Service
- C. Distributed Binary Model Server
- D. Database Mapping Syntax

Question 2. Which of the following is a function of a DBMS?

- A. Creating, storing, querying, updating, and protecting data
- B. Designing graphical user interfaces
- C. Compiling application source code
- D. Completely replacing the operating system

Section Outline

- 1 Overview
- 2 Basic Concepts
- 3 Criteria for Selecting a DBMS**
- 4 Common Database Types
- 5 Database Management and Operations
- 6 Architectural Patterns Affecting Database Choice
- 7 Comparison and DBMS Selection Process
- 8 DBMS Selection Considerations

Overview of the Main Criteria

When choosing a database for a system, consider:

1. Scalability.
2. Data consistency.
3. Query patterns and workload.
4. Operational complexity.
5. Recoverability and availability.
6. Performance requirements.

Criterion 1: Scalability

Scalability indicates whether the system can handle increasing data volume or a growing number of users.

There are two main directions for scaling:

Vertical Scaling

Increasing the resources of an existing server.

Horizontal Scaling

Adding more servers to the system to distribute data or load.

Vertical Scaling

Vertical scaling means increasing the resources of an existing server.

Examples:

- Adding more CPU.
- Adding more RAM.
- Increasing disk capacity.
- Using a more powerful server.

Remember

Relational databases such as MySQL, PostgreSQL, and SQL Server are often suitable for vertical scaling.

Horizontal Scaling

Horizontal scaling means adding more servers to the system.

Examples:

- Splitting data across multiple machines.
- Distributing query load.
- Using a server cluster.

Remember

NoSQL databases are often designed to support horizontal scaling well.

Criterion 2: Data Consistency

Data consistency indicates whether data is accurate and synchronized at all times.

Banking Example

When account A transfers money to account B, the amount must be deducted from A and added to B. These two operations must occur together.

If money is deducted from A but not added to B, the data becomes incorrect.

Strong Consistency and Eventual Consistency

Strong Consistency

Financial, accounting, and banking systems usually require strong consistency and ACID-compliant transactions.

Eventual Consistency

Some systems can tolerate data becoming consistent after a short delay, such as social networks, recommendation systems, logging systems, or view-count analytics.

Criterion 3: Query Patterns and Workload

Before choosing a DBMS, we need to understand what the system mainly does with its data.

Common workload patterns include:

- Read-heavy workloads.
- Write-heavy workloads.
- Balanced read and write workloads.
- Complex queries.
- Simple queries requiring very high speed.

Query Patterns and Suitable Database Types

System Characteristic	Suitable Database Type
Requires many complex JOIN operations	Relational database
Requires reporting, statistics, and analytics	Relational database or columnar database
Requires very fast key-based access	Key-value store
Uses flexible JSON data	Document database
Uses sensor data over time	Time-series database
Contains many complex relationships	Graph database

Criterion 4: Operational Complexity

A database system not only needs to be well designed; it also needs to be operated well.

Operational tasks include:

- Backing up data.
- Recovering data.
- Upgrading versions.
- Monitoring performance.
- Managing user permissions.
- Encrypting data.
- Setting up replication.
- Handling incidents.

If the technical team has limited database administration experience, a **managed database** should be considered.

Criterion 5: Recoverability and Availability

Some systems must run continuously:

- Digital banking.
- E-commerce.
- Ticket booking systems.
- Hospital management systems.
- Real-time logistics systems.

Such systems need automatic backups, automatic failover, data replication, multi-region support, and disaster recovery capability.

Criterion 6: Performance Requirements

Performance can be evaluated through:

- Low latency.
- High throughput.
- Data write speed.
- Data read speed.
- Ability to process concurrent queries.

Examples

Cache systems require very low latency; IoT systems need very high write speed; BI systems require large analytical queries; transaction systems require consistency and reliability.

Quick Quiz: DBMS Selection Criteria

Question 1. What does horizontal scaling mean?

- A. Adding more CPU and RAM to an existing server
- B. Adding more servers to distribute load or data
- C. Reducing the number of system users
- D. Moving all data into text files

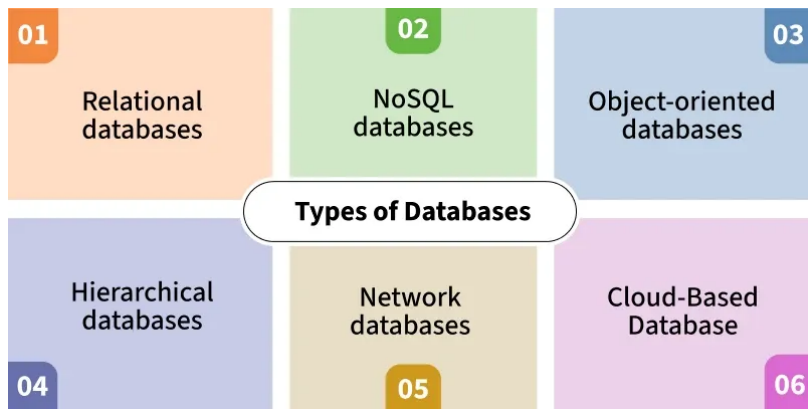
Question 2. What property is usually required in a banking system?

- A. Strong consistency and ACID transactions
- B. Eventual consistency for all transactions only
- C. No need for data integrity control
- D. No need for backups

Section Outline

- 1 Overview
- 2 Basic Concepts
- 3 Criteria for Selecting a DBMS
- 4 Common Database Types**
 - Relational Database
 - Document Database
 - Key-Value Store
 - Time-Series Database
 - Columnar Database
 - Graph Database
- 5 Database Management and Operations

Figure 2. Common Database Types



Common database types.

Database Types Covered in This Lesson

- Relational database.
- Document database.
- Key-value store.
- Time-series database.
- Columnar database.
- Graph database.

Quick Quiz: Common Database Types

Question 1. Tabular data with primary keys and foreign keys is usually suitable for which database type?

- A. Relational database
- B. Time-series database
- C. Graph database
- D. Key-value store

Question 2. Flexible JSON data is usually suitable for which database type?

- A. Document database
- B. Columnar database
- C. Relational database in every case
- D. Time-series database

Relational Database: Concept

A **Relational Database Management System**, or **RDBMS**, stores data in tables.

Each table includes:

- Rows.
- Columns.
- Primary keys.
- Foreign keys.
- Data constraints.

RDBMSs commonly use SQL to query data.

Example: Students Table

student_id	full_name	major
1	Nguyen An	Data Science
2	Tran Binh	Logistics
3	Le Chi	Information Systems

Example SQL query:

SQL

```
SELECT full_name, major FROM Students WHERE major = 'Data Science';
```

When Should We Use an RDBMS?

Use an RDBMS when:

- The data has a clear structure.
- Data integrity must be enforced.
- ACID transactions are required.
- JOIN operations across multiple tables are needed.
- Reporting and complex queries are required.
- The system involves finance, accounting, order management, or student management.

Examples Suitable for RDBMSs

- Banking systems.
- Student management systems.
- Sales systems.
- Inventory management systems.
- Accounting systems.

When Should We Use a Managed RDBMS?

Use a managed RDBMS service when:

- High availability is required.
- Automatic backups are desired.
- Point-in-time recovery is needed.
- The team does not want to manage version upgrades manually.
- Built-in security, encryption, and monitoring are required.
- The team wants to reduce operational burden.

Examples and When Not to Use Managed RDBMS

Examples

Amazon RDS, Google Cloud SQL, Azure SQL Database.

Managed RDBMS may not be suitable if

- Deep control over the database engine is required.
- Data must be stored on premises.
- Legal constraints do not allow data to be stored in the cloud.
- Cloud costs do not fit the budget.

Document Database: Concept

A **document database** stores data as documents, usually in JSON or BSON format.

Unlike relational databases, document databases do not require every record to have the same structure.

Example JSON Document

```
{
  "student_id": 1,
  "full_name": "Nguyen An",
  "major": "Data Science",
  "skills": ["Python", "SQL", "Machine Learning"],
  "address": {
    "city": "Hanoi",
    "district": "Hoan Kiem"
  }
}
```

When Should We Use a Document Database?

Use a document database when:

- The data structure is flexible.
- Data is naturally represented as JSON.
- The schema changes frequently.
- The data contains nested structures.
- The application evolves quickly and requires flexible data modeling.

Suitable Examples and Managed Document Databases

Suitable Examples

User profiles, product catalogs, article content, mobile applications, and content management systems.

Managed Document Databases

MongoDB Atlas, Amazon DocumentDB, Couchbase Capella.

A managed document database is useful when fast scaling, replication, sharding, backup, multi-region deployment, access control, and monitoring are needed.

Key-Value Store: Concept

A **key-value database** stores data as pairs of keys and values.
This database type is simple but provides very fast access speed.

Example Key-Value Data

```
user:1001 -> "Nguyen An"  
cart:1001 -> ["book", "laptop", "mouse"]  
session:abc123 -> "active"
```

When Should We Use a Key-Value Store?

Use a key-value store when:

- Extremely fast data access is required.
- Data is mainly queried by key.
- User sessions need to be stored.
- Data caching is required.
- Game leaderboards are needed.
- Millions of requests per second must be processed.

Suitable Examples and Systems

Suitable Examples

Product cache, user sessions, temporary shopping carts, leaderboards, and view counters.

Example Systems

Redis, Memcached, Amazon ElastiCache, Azure Cache for Redis, Google Memorystore.

Time-Series Database: Concept

A **time-series database** is designed for data associated with time.

Each record usually contains:

- A timestamp.
- A measured value.
- A data source.

Example Time-Series Data

2026-05-31	10:00:00		sensor_01		temperature		32.5
2026-05-31	10:01:00		sensor_01		temperature		32.7
2026-05-31	10:02:00		sensor_01		temperature		32.6

When Should We Use a Time-Series Database?

Use a time-series database for:

- Sensor data.
- IoT data.
- System logs.
- Monitoring metrics.
- Financial data over time.
- Machine operation data.
- Real-time dashboards.
- Historical time-based analysis.

Suitable Examples and Systems

Suitable Examples

Cold-storage temperature monitoring, server CPU/RAM/disk monitoring, truck location tracking over time, stock price analysis, and electricity consumption analysis.

Example Systems

InfluxDB Cloud, Amazon Timestream, Azure Data Explorer, TimescaleDB.

Columnar Database: Concept

A **columnar database** stores data by columns instead of by rows.

If the goal is to compute total revenue, the system only needs to read the amount column. Therefore, columnar databases are highly efficient for large-scale data analytics.

Example: Sales Table

order_id	customer_id	amount	date
1	C01	100	2026-05-01
2	C02	200	2026-05-02
3	C03	150	2026-05-03

For revenue analytics, column-based reading reduces the amount of data that must be scanned.

When Should We Use a Columnar Database?

Use a columnar database when:

- Large-scale data analysis is required.
- BI or data warehousing is needed.
- Queries are mainly read and aggregation queries.
- Statistical computation over millions or billions of rows is required.
- Storage and compute should be separated.

Suitable Examples and Systems

Suitable Examples

Revenue reporting, customer behavior analysis, management dashboards, enterprise data warehouses, and logistics data analysis.

Example Systems

Google BigQuery, Amazon Redshift, Snowflake, Azure Synapse Analytics.

Graph Database: Concept

A **graph database** represents data using:

- **Nodes**: entities.
- **Edges**: relationships.
- **Properties**: attributes.

Graph databases are suitable when relationships among objects are more important than individual objects alone.

Example Graph Data

```
Minh --friend_of--> An  
An --works_at--> Company A  
Company A --located_in--> Hanoi
```

When Should We Use a Graph Database?

Use a graph database when:

- Data has many complex relationships.
- Path or network queries are needed.
- Link analysis is required.
- Fraud detection is needed.
- Recommendation systems need to be built.

Suitable Examples and Systems

Suitable Examples

Social networks, friend recommendations, financial fraud analysis, product recommendations, knowledge management, and supply chain dependency analysis.

Example Systems

Neo4j Aura, Amazon Neptune, Azure Cosmos DB Gremlin API.

Section Outline

- 1 Overview
- 2 Basic Concepts
- 3 Criteria for Selecting a DBMS
- 4 Common Database Types
- 5 Database Management and Operations**
 - Managed Database
 - When Should We Choose a Managed Database?
 - When Should We Self-Manage a Database?
- 6 Architectural Patterns Affecting Database Choice

What Is a Managed Database?

A **managed database** is a database service managed by a cloud provider on behalf of the technical team.

The provider may support:

- Automatic backups.
- Data recovery.
- Version upgrades.
- Resource scaling.
- Data replication.
- Performance monitoring.
- Data encryption.
- Failover.

Examples of Managed Databases

- Amazon RDS.
- Google Cloud SQL.
- Azure Cosmos DB.
- MongoDB Atlas.
- Amazon Timestream.
- Snowflake.

When Should We Choose a Managed Database?

Choose a managed database when:

- The technical team is small.
- The team does not want to spend too much time on operations.
- High availability is required.
- Automatic backup and recovery are needed.
- Integrated monitoring is required.
- Security and compliance are important.
- The team wants to focus on application development rather than infrastructure administration.

When Should We Self-Manage a Database?

Self-management may be suitable when:

- Deep control over system configuration is required.
- Special modules must be installed.
- Data must be stored on premises.
- The organization has a strong DBA team.
- Cost optimization is needed in specific cases.
- There are special security or legal requirements.

Section Outline

- 1 Overview
- 2 Basic Concepts
- 3 Criteria for Selecting a DBMS
- 4 Common Database Types
- 5 Database Management and Operations
- 6 Architectural Patterns Affecting Database Choice**
 - Polyglot Persistence
 - CQRS
 - Event Sourcing
 - Microservices

Polyglot Persistence

Polyglot persistence means that one system uses multiple database types for different needs.

E-commerce Example

- PostgreSQL for orders and payments.
- MongoDB for product catalogs.
- Redis for caching.
- Elasticsearch for search.
- Snowflake for data analytics.

The main idea: the entire system does not have to use only one database type.

CQRS

CQRS stands for **Command Query Responsibility Segregation**.

Main idea:

- Separate the write side.
- Separate the read side.

Example

An SQL database can be used for transactional writes, while a columnar database can be used for reporting and analytics.

CQRS is suitable when the system has significantly different read and write requirements.

Event Sourcing

Event sourcing stores state-changing events instead of storing only the final state.

Order Example

OrderCreated	PaymentConfirmed	OrderPacked	OrderShipped
OrderDelivered			

- Provides a complete change history.
- Makes auditing easier.
- Allows the system state to be reconstructed at a past point in time.
- Fits systems that require traceability.

Microservices

In a microservices architecture, each service may manage its own database.

Example

- The user service uses PostgreSQL.
- The product service uses MongoDB.
- The cache service uses Redis.
- The analytics service uses BigQuery.

This helps each service choose the database that best fits its business responsibility.

Section Outline

- 1 Overview
- 2 Basic Concepts
- 3 Criteria for Selecting a DBMS
- 4 Common Database Types
- 5 Database Management and Operations
- 6 Architectural Patterns Affecting Database Choice
- 7 Comparison and DBMS Selection Process**
 - Comparison Table
 - Five Step Selection Process

Quick Comparison of Database Types

Database Type	Main Data Form	Suitable For	Examples
Relational database	Tables, rows, columns	Transactions, structured data, JOINS	MySQL, PostgreSQL
Document database	JSON, BSON	Flexible data, changing schema	MongoDB
Key-value store	Key-value pairs	Cache, sessions, extremely fast access	Redis
Time-series database	Time-based data	IoT, logs, metrics, sensors	InfluxDB
Columnar database	Column-based storage	BI, OLAP, data warehouses	BigQuery, Snowflake
Graph database	Nodes and edges	Complex relationships, social networks, fraud detection	Neo4j

DBMS Selection Process

A five-step process can be used:

1. Identify the nature of the data.
2. Identify the query pattern.
3. Identify consistency requirements.
4. Identify scalability requirements.
5. Identify operational capability.

Step 1: Identify the Nature of the Data

Questions to answer:

- Does the data have a fixed structure?
- Is the data tabular?
- Is the data in JSON format?
- Is the data time-dependent?
- Does the data contain many complex relationships?

Step 2: Identify the Query Pattern

Questions to answer:

- Are many table JOINS needed?
- Is fast key-based querying required?
- Is large-scale data analysis required?
- Are multi-level relationship queries required?
- Is a real-time dashboard required?

Step 3: Identify Consistency Requirements

Questions to answer:

- Does the system require ACID transactions?
- Can the system accept eventual consistency?
- Would temporary data inconsistency cause serious consequences?

Step 4: Identify Scalability Requirements

Questions to answer:

- Does the system need vertical or horizontal scaling?
- Is the data growing rapidly?
- Can the number of users increase suddenly?
- Is multi-region deployment required?

Step 5: Identify Operational Capability

Questions to answer:

- Does the team have database administration experience?
- Is there a dedicated DBA?
- Is automatic backup required?
- Is integrated monitoring required?
- Should a managed database be used?

Section Outline

- 1 Overview
- 2 Basic Concepts
- 3 Criteria for Selecting a DBMS
- 4 Common Database Types
- 5 Database Management and Operations
- 6 Architectural Patterns Affecting Database Choice
- 7 Comparison and DBMS Selection Process
- 8 DBMS Selection Guidelines

Scenario 1: Student Management System

Characteristics

- Data has a clear structure.
- There are multiple tables such as students, classes, courses, and grades.
- Data JOINS are required.
- Data integrity must be ensured.

Suitable Choice

PostgreSQL or MySQL.

Scenario 1: Reasoning

PostgreSQL or MySQL is suitable because:

- They fit relational data.
- They support SQL.
- They support primary keys and foreign keys.
- They make reporting easier.

Scenario 2: Social Network Application

Characteristics

- User data can be flexible.
- The system contains posts, comments, and interactions.
- There are friendship and following relationships.
- Large-scale expansion is required.

Possible Combined Choice

MongoDB for content; Redis for cache; a graph database for friendship relationships; a columnar database for analytics.

Scenario 3: IoT Cold-Storage Temperature Monitoring

Characteristics

- Data is continuously written over time.
- Each record has a timestamp.
- A real-time dashboard is needed.
- Historical data storage and aggregation are required.

Suitable Choice

InfluxDB or Amazon Timestream.

Scenario 3: Reasoning

InfluxDB or Amazon Timestream is suitable because:

- They are optimized for time-series data.
- They support retention policies.
- They support time-based data analysis.
- They fit real-time monitoring dashboards.

Scenario 4: E-Commerce System

Characteristics

- Orders require high consistency.
- Product information can be flexible.
- Cache is needed to improve speed.
- Revenue analytics is required.

Combined Choice

PostgreSQL for orders; MongoDB for product catalogs; Redis for cache; BigQuery or Snowflake for analytics.

Section Outline

- 1 Overview
- 2 Basic Concepts
- 3 Criteria for Selecting a DBMS
- 4 Common Database Types
- 5 Database Management and Operations
- 6 Architectural Patterns Affecting Database Choice
- 7 Comparison and DBMS Selection Process
- 8 DBMS Selection Considerations

Implementation Notes

When deploying a database system, pay attention to:

- Designing backups from the beginning.
- Having a disaster recovery plan.
- Monitoring query performance.
- Monitoring replication errors.
- Designing schemas that can evolve.
- Avoiding premature optimization before understanding the workload.
- Not choosing a database only because it is trendy.
- Prioritizing simple solutions if the problem is not yet too complex.

Lesson Summary

In this lesson, we learned that:

- DBMS selection is an important architectural decision.
- RDBMSs fit structured data and strong consistency requirements.
- Document databases fit flexible JSON-like data.
- Key-value stores fit caching and extremely fast access.
- Time-series databases fit time-based data.
- Columnar databases fit large-scale analytics.
- Graph databases fit data with many complex relationships.
- Managed databases reduce operational burden.
- Large systems may use multiple database types.

No single database type fits every problem

A good choice must be based on data characteristics, query patterns, consistency requirements, performance, scalability, operational capability, cost, and the system's long-term direction.

In modern systems, combining multiple database types is common. The key is to use each database type according to its strengths.

Keywords

- DBMS
- Relational Database
- Document Database
- Key-Value Store
- Time-Series Database
- Columnar Database
- Graph Database
- Managed Database
- ACID
- Eventual Consistency
- Scalability
- Vertical Scaling
- Horizontal Scaling
- Backup
- Polyglot Persistence
- CQRS
- Event Sourcing
- Microservices
- Failover
- Workload
- Monitoring

Section Outline

- 1 Overview
- 2 Basic Concepts
- 3 Criteria for Selecting a DBMS
- 4 Common Database Types
- 5 Database Management and Operations
- 6 Architectural Patterns Affecting Database Choice
- 7 Comparison and DBMS Selection Process
- 8 DBMS Selection Guidelines

Review Questions I

1. Why should we not choose a DBMS only because it is popular?
2. Distinguish vertical scaling from horizontal scaling.
3. Which database type is suitable for a banking system? Why?
4. When should a document database be used?
5. In what situations is Redis commonly used?
6. What type of data is suitable for a time-series database?
7. How is a columnar database different from a row-oriented database?
8. What types of problems are suitable for graph databases?
9. What advantages does a managed database provide?
10. What is polyglot persistence?

Short Review Quiz I

Question 1. When choosing a DBMS, which factor should be considered first?

- A. The provider's logo
- B. The nature of the data and how the system queries it
- C. Which database is most popular on social media
- D. The programming language of the user interface

Question 2. Which database type is usually preferred for a banking system?

- A. Relational database
- B. Key-value store
- C. Graph database
- D. Time-series database

Question 3. If data has a flexible JSON structure and the schema changes frequently, what is a suitable choice?

Short Review Quiz II

- A. Document database
- B. Columnar database
- C. Time-series database
- D. Key-value store

Question 4. Which situation is Redis most suitable for?

- A. Complex JOIN queries across many tables
- B. Cache, sessions, or very fast key-based access
- C. Historical analysis over billions of rows
- D. Modeling multi-level social relationships

Question 5. IoT sensor data continuously sent with timestamps should prioritize which database type?

- A. Graph database

Short Review Quiz III

- B. Time-series database
- C. Document database
- D. Relational database

Question 6. What need is a columnar database most suitable for?

- A. Storing login sessions
- B. Analyzing, aggregating, and reporting large-scale data
- C. Storing friendship relationships in a social network
- D. Managing ACID banking transactions

Question 7. What is the main strength of graph databases?

- A. Querying and analyzing complex relationships among objects
- B. Storing large image files

Short Review Quiz IV

- C. Completely replacing the operating system
- D. Only storing fixed tabular data

Question 8. What operational work can a managed database reduce for the technical team?

- A. Designing the user interface
- B. Writing all business logic
- C. Backups, monitoring, version upgrades, and failover
- D. Eliminating the need for data security

Application Exercises I

Exercise 1. A startup is building an online order management application. The system includes customer management, product management, order management, payment, revenue reporting, and product search. Propose a suitable database type for each function.

Exercise 2. A logistics company wants to store truck location data in real time. Each truck sends GPS data every 10 seconds. Answer the following:

- What database type should be used?
- Why?
- Should a traditional RDBMS be used as the only database?
- What additional tool is needed for a real-time dashboard?

Exercise 3. A social networking system needs to store user profiles, posts, comments, friendship relationships, friend recommendations, and view statistics. Propose a data storage architecture that uses multiple database types.

Additional Multiple-Choice Questions

Question 1. Which database type is most suitable for clearly structured data that requires complex JOINS?

- A. Key-value store
- B. Relational database
- C. Time-series database
- D. Graph database

Question 2. What is Redis commonly used for?

- A. Large-scale data analysis
- B. Storing complex social relationships
- C. Cache and sessions
- D. Column-based data storage

Additional Multiple-Choice Questions

Question 3. Sensor data over time is most suitable for which database type?

- A. Time-series database
- B. Graph database
- C. Document database
- D. Relational database

Question 4. MongoDB is a popular example of which database type?

- A. Columnar database
- B. Document database
- C. Key-value store
- D. Time-series database

Question 5. Graph databases are most suitable for which problem?

- A. Storing user sessions
- B. Analyzing complex relationships
- C. Computing total monthly revenue

Thank you!

Questions and Discussion